Classification non supervisée (Clustering)

Méthodes de type K-means et DBSCAN

Cathy Maugis-Rabusseau

4modIA / INSA Toulouse & ENSEEIHT

2026-2027

Introduction

- Données : On observe n individus ($\Omega = \{1, \dots, n\}$) décrits par p variables

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \text{ avec } x_i = (x_{i1}, \dots, x_{ip}) \in \mathcal{X}$$

- Soit d une distance euclidienne

- But : trouver une partition de l'ensemble des individus telle que l'inertie intra-classe soit minimale

$$I_{intra} = \sum_{k=1}^{K} \sum_{i \in \mathcal{C}_k} d(x_i, m_k)^2$$

Chapitre 2 - Méthodes de type K-means et DBSCAN

1. Méthodes de type K-means
2. Méthodes de type K-médoïdes
3. Choix des noyaux initiaux
4. Choix du nombre de classes
5. DBSCAN

Principe général des méthodes de type K-means

- Initialisation :
  - choix du nombre de classes K
  - choix de K noyaux initiaux $c_1^{(0)}, \dots, c_K^{(0)}$
- On itère les deux étapes suivantes : à l'itération t
  - Affectation des individus à la classe la plus proche :

$$i \in \mathcal{C}_k^{(t)} \text{ si } d\left(x_i, c_k^{(t-1)}\right) = \min_{1 \leq k' \leq K} d\left(x_i, c_{k'}^{(t-1)}\right)$$

  - Mise à jour des noyaux : $c_1^{(t)}, \dots, c_K^{(t)}$
- Arrêt de l'algorithme quand la classification n'est plus modifiée

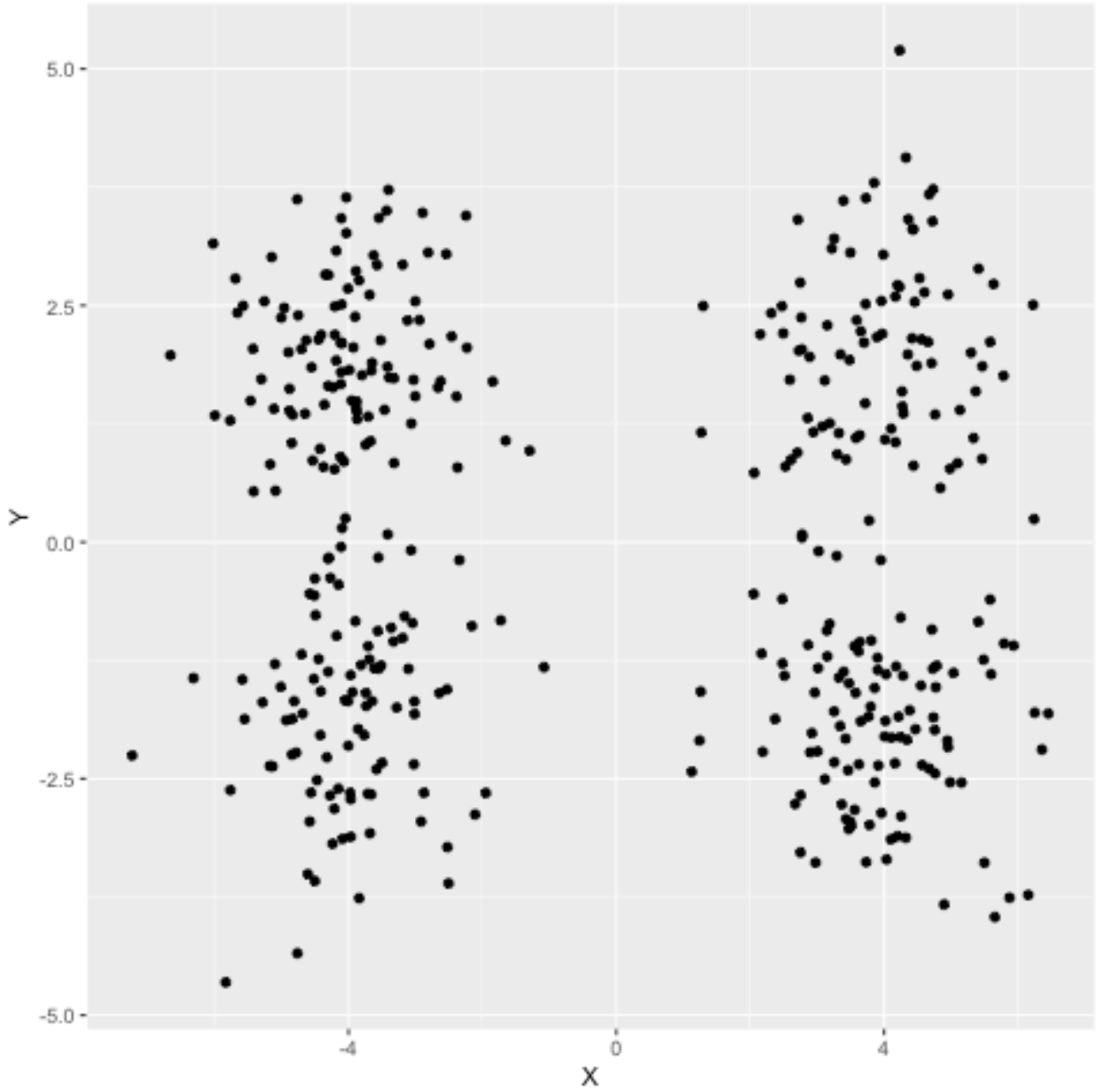
Méthode des centres mobiles (Forgy 1965)

- $c_1^{(0)}, \dots, c_K^{(0)}$ sont choisis au hasard (sans remise) parmi les x_i de $\mathbf{X}$
- À l'itération t :
  - $i \in \mathcal{C}_k^{(t)}$ si $d\left(x_i, c_k^{(t-1)}\right) = \min_{1 \leq k' \leq K} d\left(x_i, c_{k'}^{(t-1)}\right)$
  - Mise à jour des noyaux :

$$\forall k \in \{1, \dots, K\},\ c_k^{(t)} = \frac{1}{|\mathcal{C}_k^{(t)}|} \sum_{i \in \mathcal{C}_k^{(t)}} x_i$$

- Arrêt de l'algorithme quand 2 itérations successives fournissent la même classification

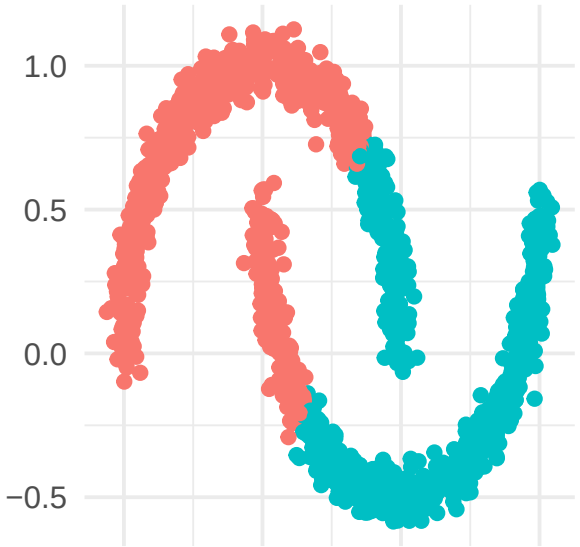
Méthode des centres mobiles (Forgy 1965)

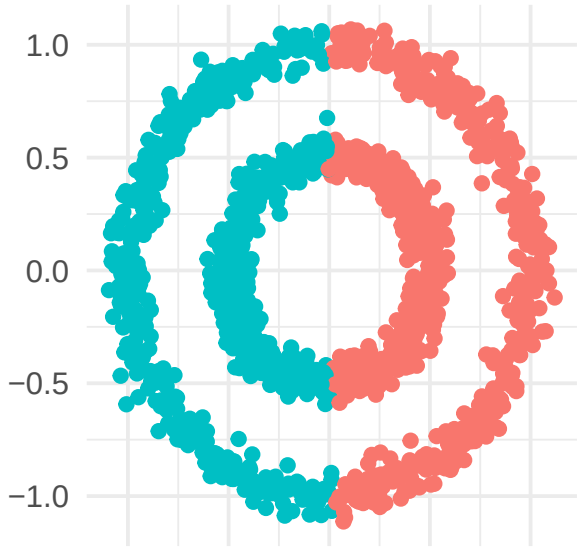


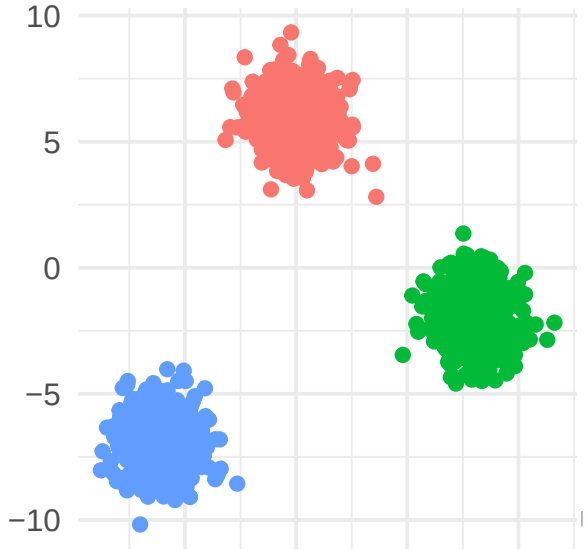
Points forts / faibles

- Avantages :
  - Relativement efficace (rapide)
  - Tend à réduire l'inertie intra-classe à chaque itération
  - Forme des classes compactes, bien séparées

- Inconvénients :
  - Influence du choix des noyaux initiaux
  - Convergence vers un minimum local
  - Nécessite la notion de centre de gravité
  - Influence des outliers
  - Non adapté pour des classes non convexes







Propriétés

Proposition

L'inertie intraclasse $I_{\text{intra}}(\mathcal{P}_K^{(t)})$ décroît à chaque étape.
$\Rightarrow$ convergence vers un minimum local de l'inertie intra-classe.

Preuve: Les deux points clés sont :

- si i passe de $\mathcal{C}_k^{(t-1)}$ à $\mathcal{C}_{k'}^{(t)}$ alors

$$d\left(x_i, c_{k'}^{(t-1)}\right)^2 \leq d\left(x_i, c_k^{(t-1)}\right)^2$$

- $c_k^{(t)}$ étant le centre de $\mathcal{C}_k^{(t)}$

$$\sum_{i \in \mathcal{C}_k^{(t)}} d\left(x_i, c_k^{(t)}\right)^2 \leq \sum_{i \in \mathcal{C}_k^{(t)}} d\left(x_i, c_k^{(t-1)}\right)^2$$

Variante : K-means (McQueen 1967)

- Initialisation :
  - choix de K
  - $c_1^{(0)}, \dots, c_K^{(0)}$ sont tirés aléatoirement parmi les n points observés
- À chaque itération t
  - tirage au hasard d'un nouveau point x_i de $\mathbf{X}$
  - détermination du noyau $c_k^{(t-1)}$ le plus proche de x_i
  - mise à jour du noyau :

$$c_k^{(t)} = \frac{x_i + |\mathcal{C}_k^{(t-1)}| c_k^{(t-1)}}{|\mathcal{C}_k^{(t-1)}| + 1}$$

Nuées dynamiques (Diday 1971)

- Initialisation : choix de K sous-ensembles disjoints $N_1^{(0)}, \ldots, N_K^{(0)}$ de cardinal q chacun (q fixé)
- À l'itération t :
  - On affecte les individus de Ω

$$\mathcal{C}_k^{(t)} = \left\{ i \in \Omega;\ D\left(x_i, N_k^{(t-1)}\right) \leq D\left(x_i, N_r^{(t-1)}\right)\ \forall r \neq k \right\}$$

avec $D(A,B) = \sum_{x \in A} \sum_{y \in B} d(x,y)$ pour toutes parties A, B de $\mathcal{X}$

  - Mise à jour des noyaux $(N_k^{(t)})_{1 \leq k \leq K}$: $N_k^{(t)}$ est le sous-ensemble de cardinal q qui minimise

$$\sum_{x \in N_k^{(t)}} D\left(x, \mathcal{C}_k^{(t)}\right).$$

Quelques commandes

- Avec R 

`km=kmeans(x, centers=, iter.max=, nstart=, algorithm=)`

- *centers*: vecteur des noyaux initiaux ou nombre de classes
- *iter.max*: nombre d'itérations maximal
- *nstart*: nombre d'initialisations aléatoires (on garde la partition de plus faible inertie intra-classe)
- *algorithm*: "Hartigan-Wong" par défaut, "Lloyd" ou "Forgy" (centres mobiles), "MacQueen" (K-means)

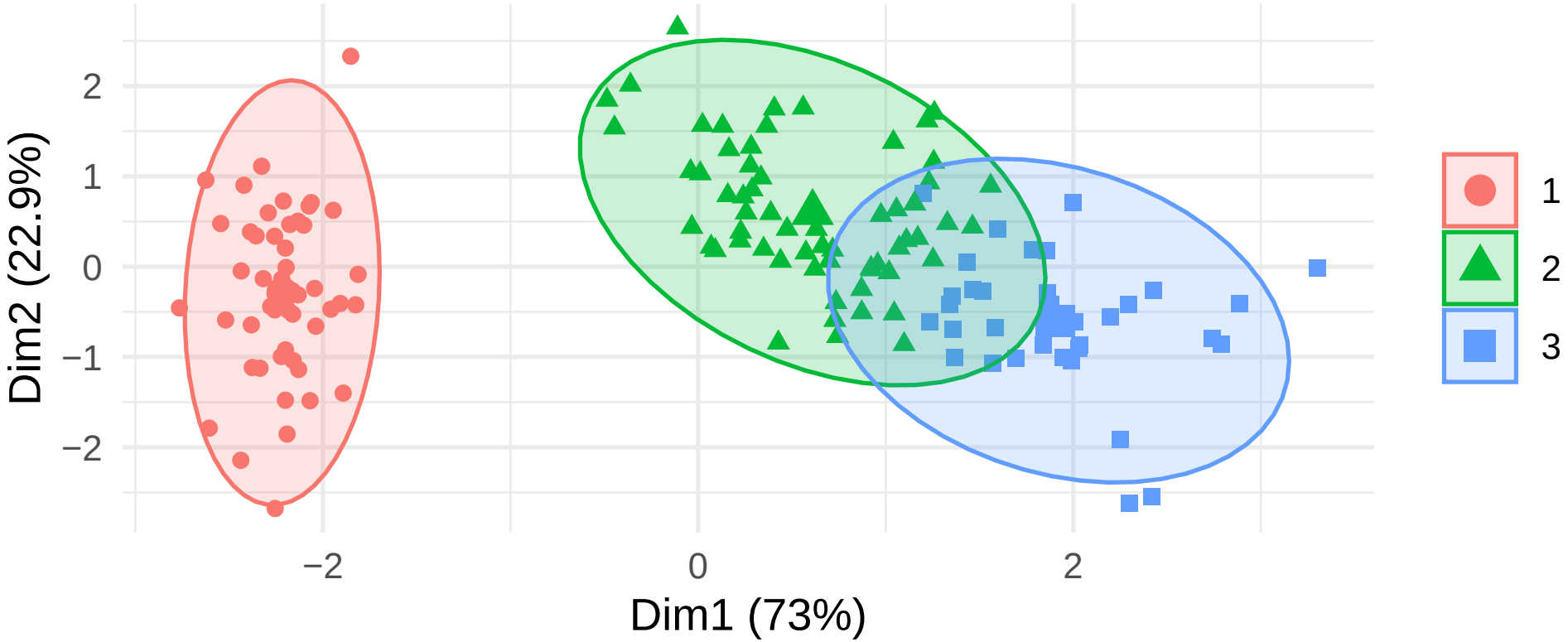
`km$cluster`, `km$centers`, `km$withinss`, `km$size`

- Avec Python 

`kmeans=sklearn.cluster.KMeans(n_clusters=,...); kmeans.fit(data)`
`kmeans.labels_`, `kmeans.cluster_centers_`, ...

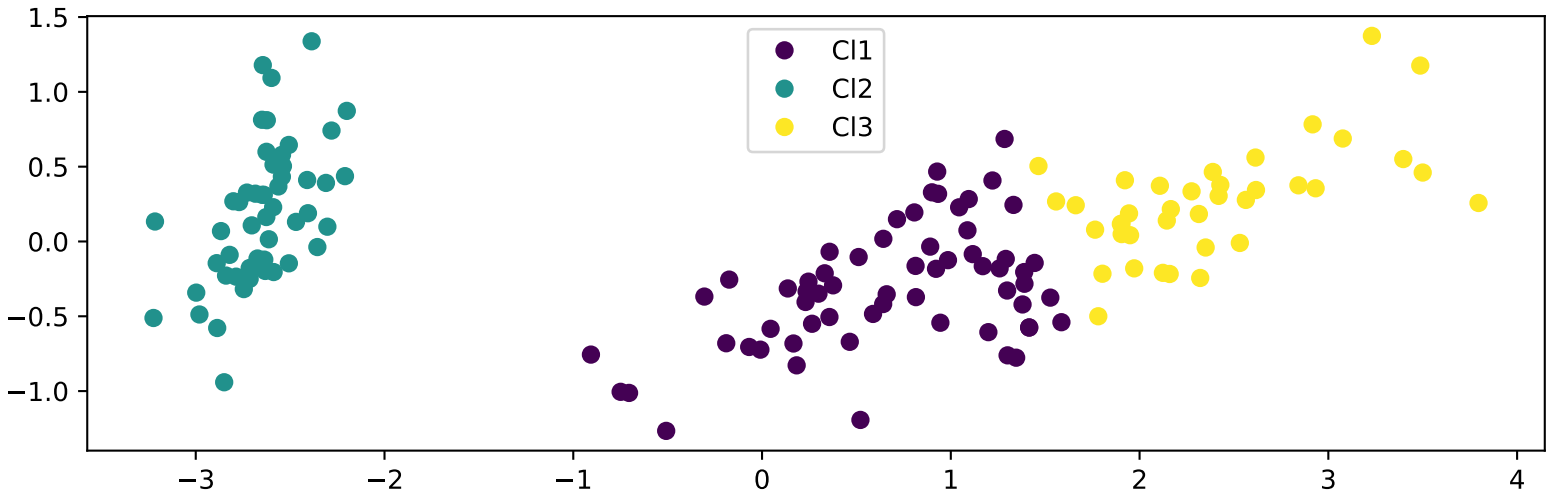
Exemple - Données iris

```
data(iris)
set.seed(1234)
kmiris=kmeans(iris[,1:4],centers=3)
fviz_cluster(kmiris,data=iris[,1:4],ellipse.type="norm",
             labelsize=8,geom=c("point"))+
  ggtitle("")+themeCours
```



Exemple - Données iris

```
from sklearn.cluster import KMeans
pyiris=r.iris
X=pyiris.iloc[:, [0, 1, 2, 3]].values
kmeans = KMeans(n_clusters = 3, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 0)
y_kmeans = kmeans.fit_predict(X)
```



Extensions des K-means

- Pour variables **qualitatives** :
  - Dans les méthodes de type K-means, chaque classe est représentée par son centre de gravité, non adapté pour des variables qualitatives
  - Algorithme K-modes (Huang 1998) : même principe que l'algorithme des centres mobiles en modifiant la dissimilarité

$$d(x_i, x_\ell) = \sum_{j=1}^{p} \frac{n_{ij} + n_{\ell j}}{n_{ij} \times n_{\ell j}} \mathbb{1}_{x_{ij} \neq x_{\ell j}}$$

    où $n_{ij} = \sharp\{v \in \{1, \dots, n\};\ x_{ij} = x_{vj}\}$. Utilisation des modes à la place des centres de gravité

- Pour variables **mixtes** : Algorithme K-prototype (Huang 1997)
- **Fuzzy c-means** : Classification "floue" (chaque individu a une "probabilité" d'appartenance à chaque classe)

Exemple des races de chiens

- Avec l'algorithme des K-modes

```
library(klaR)
set.seed(1234)
clkmodes<-kmodes(chiens[,-7],3,iter.max=100,weight=FALSE)
adjustedRandIndex(chiens[,7],clkmodes$cluster)
```

```
[1] 0.2585719
```

- Avec l'algorithme des Kmeans sur les coordonnées d'ACM

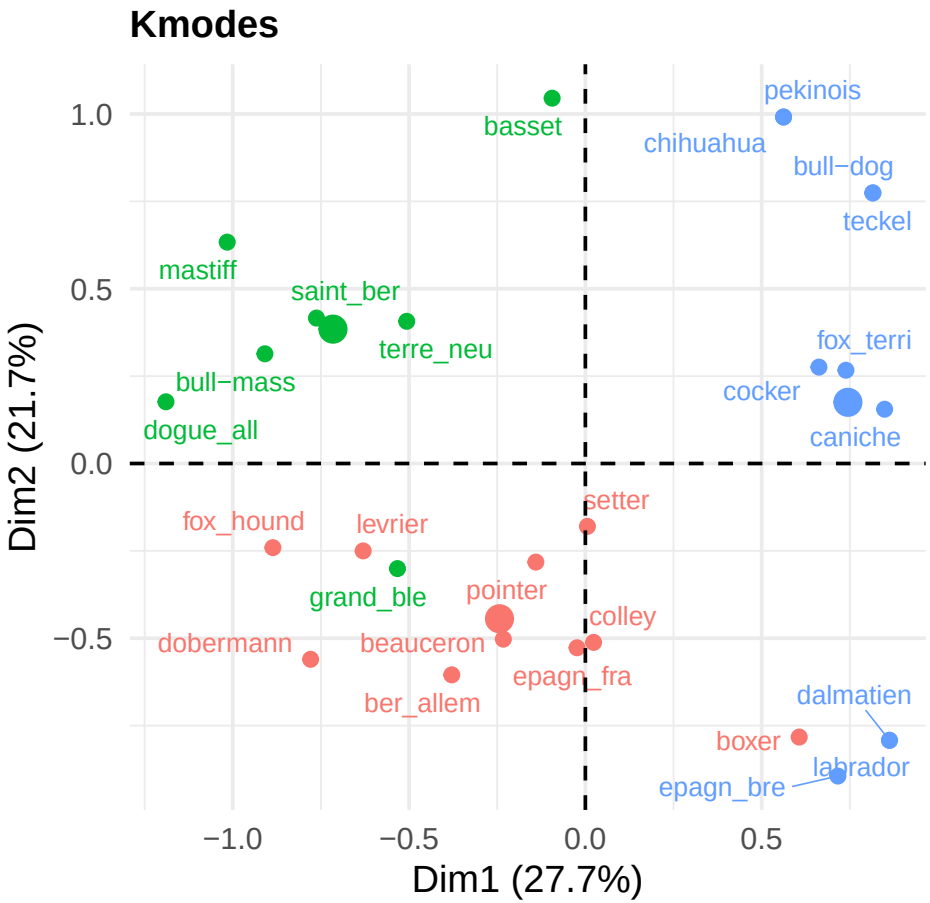
```
afcm<-MCA(chiens,quali.sup=7,graph=F)
coeff<-afcm$ind$coord
set.seed(1234)
clkmeans<-kmeans(coeff,3,nstart=50)
adjustedRandIndex(chiens[,7],clkmeans$cluster)
```

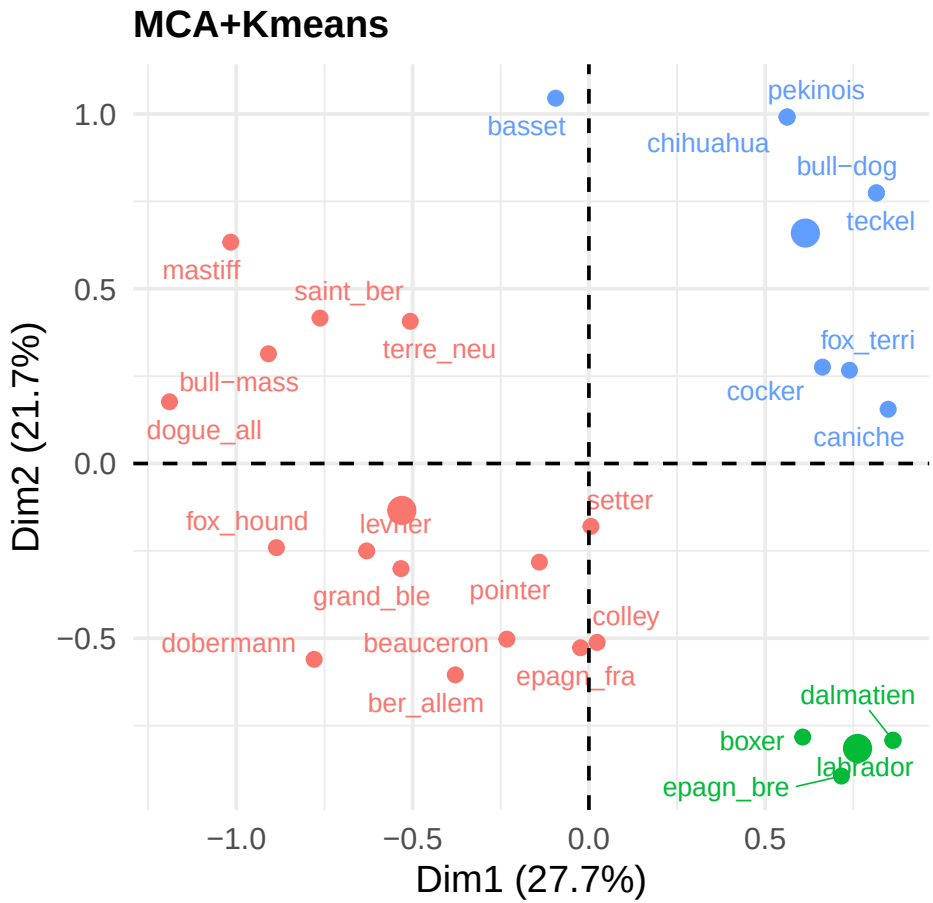
```
[1] 0.2824869
```

```
adjustedRandIndex(clkmodes$cluster,clkmeans$cluster)
```

```
[1] 0.3830416
```

Exemple des races de chiens





Chapitre 2 - Méthodes de type K-means et DBSCAN

1. Méthodes de type K-means
2. Méthodes de type K-médoïdes
3. Choix des noyaux initiaux
4. Choix du nombre de classes
5. DBSCAN

Méthode des K-médoïdes

- Idée : On remplace les centres de gravité par des **médoïdes** (des x_i de $\mathbf{X}$)
- Initialisation : choix aléatoire de K points (médoïdes) $c_1^{(0)}, \ldots, c_K^{(0)}$ parmi les n
- À chaque itération t
  - on associe chaque point x_i à son plus proche médoïde
  - pour chaque médoïde $c_k^{(t-1)}$: pour chaque point x_i qui n'est pas un médoïde, on échange x_i avec $c_k^{(t-1)}$ et on calcule le coût total de la configuration
  - on sélectionne la configuration la moins coûteuse

PAM (Partitioning Around Medoids)

Cet algorithme a été proposé par Kaufman and Rousseeuw (1990).

- Choix de K médoïdes $c_1, \ldots, c_K$ parmi les n individus
- Sélectionner au hasard un médoïde c_k et un autre objet (non médoïde) x_i
- Calculer la qualité de la nouvelle partition si les rôles de c_k et x_i sont inversés,
- Échanger c_k et x_i si la qualité est supérieure
- Et retourner en (2) jusqu'à stabilité de la qualité de la partition.

Méthode des K-médoïdes

- Chaque classe est représentée par un individu de la classe (médoïde) donc pas de limitation sur le type de variables prises en compte
- Algorithme efficace pour de petits jeux de données
- PAM est plus robuste que K-means en présence de bruit ou d'outliers (un médoïde est moins influencé par un outlier que la moyenne)

Quelques commandes

- Avec R 
  - PAM : `pam(x, k, diss = , metric = , ...)` [library(cluster)]
    metric = "euclidean" ou "manhattan", sinon x matrice de dissimilarité
  - CLARA : `clara(x, k, metric = ,...)` [library(cluster)]
    metric = "euclidean" ou "manhattan", sinon x matrice de dissimilarité
    fonction à privilégier si le nombre d'individus n est grand (travail sur des sous-échantillons)
- Avec python 
  - `sklearn_extra.cluster.KMedoids()`

Exemple des iris avec 

```
A<-pam(iris[,1:4], 3, metric = "euclidean")
A$medoids
```

```
     Sepal.Length Sepal.Width Petal.Length Petal.Width
[1,]          5.0         3.4          1.5         0.2
[2,]          6.0         2.9          4.5         1.5
[3,]          6.8         3.0          5.5         2.1
```

```
A$id.med
```

```
[1]   8  79 113
```

```
adjustedRandIndex(A$clustering,iris$Species)
```

```
[1] 0.7302383
```

```
table(iris$Species,A$clustering)
```

```
             1  2  3
  setosa     50  0  0
  versicolor  0 48  2
  virginica   0 14 36
```

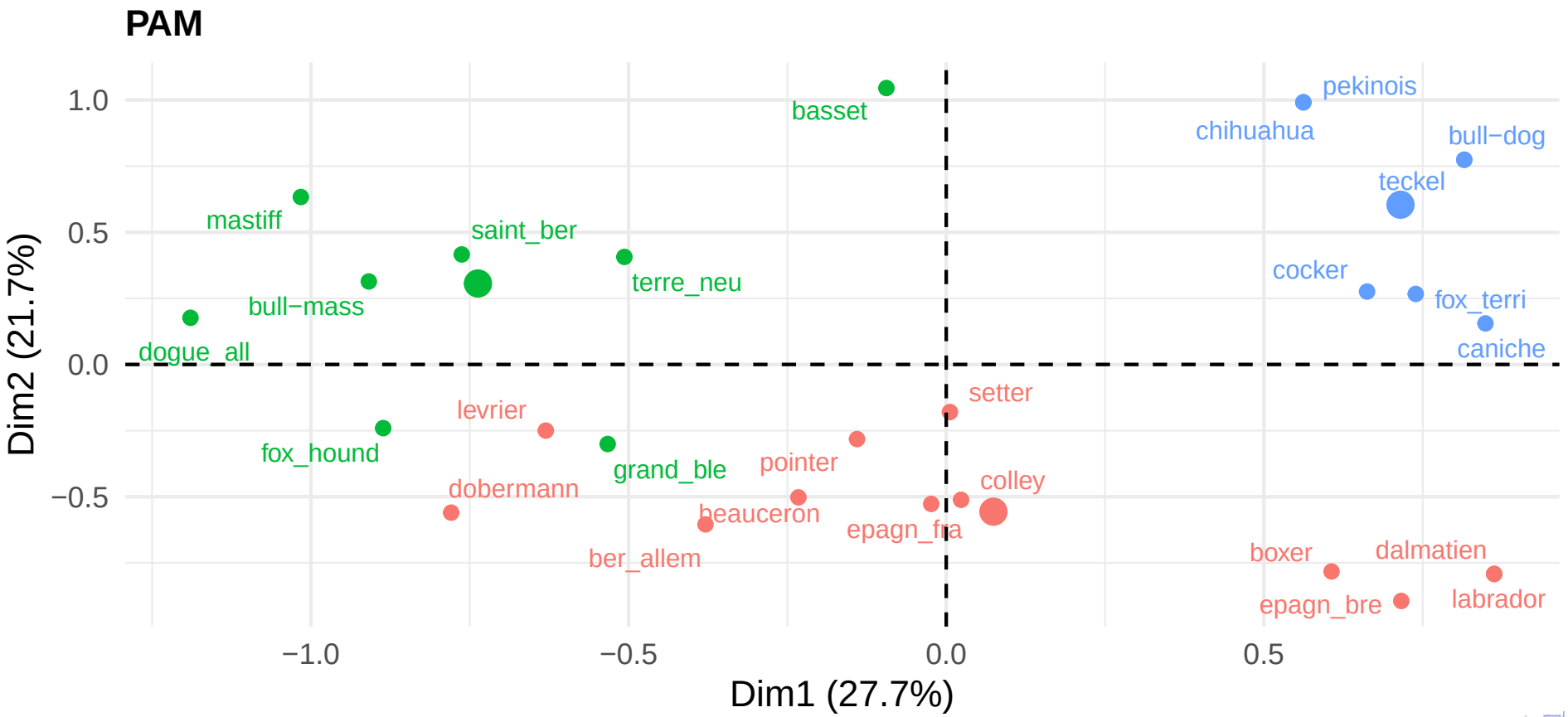
Exemple des races de chien

```
Clpam<-pam(daisy(chiens[,-7],metric="gower"),3)
adjustedRandIndex(Clpam$clustering,chiens[,7])
```

```
[1] 0.2528289
```

```
adjustedRandIndex(Clpam$clustering,clkmeans$cluster)
```

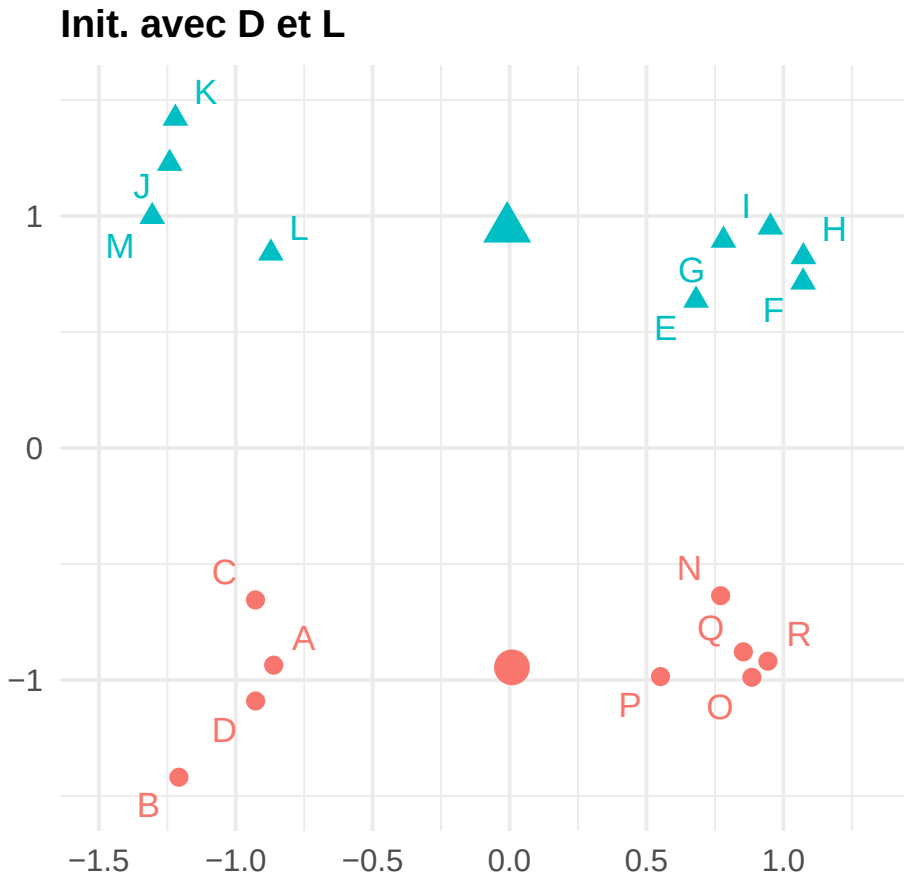
```
[1] 0.3739158
```

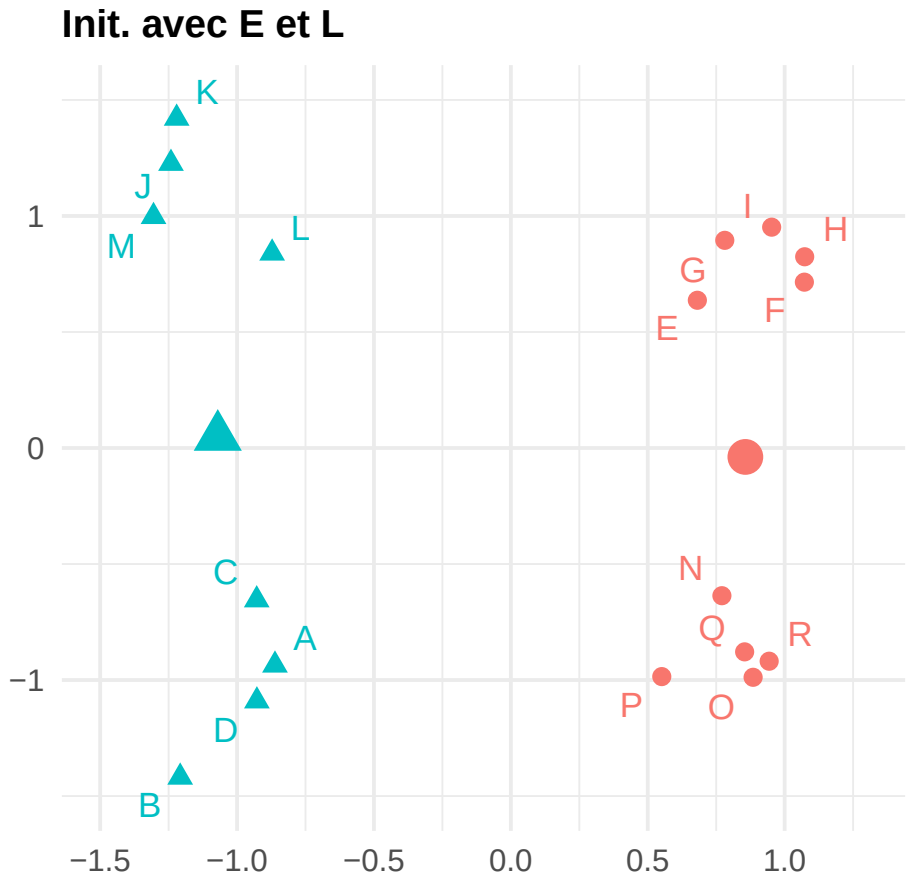


Chapitre 2 - Méthodes de type K-means et DBSCAN

1. Méthodes de type K-means
2. Méthodes de type K-médoïdes
3. Choix des noyaux initiaux
4. Choix du nombre de classes
5. DBSCAN

Initialisation des noyaux



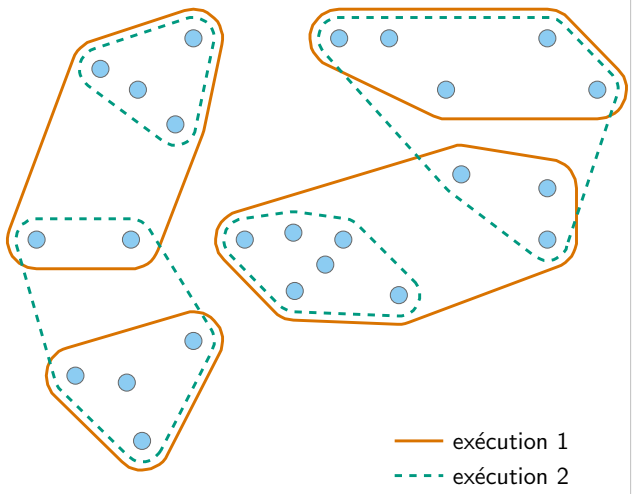


Choix des noyaux initiaux

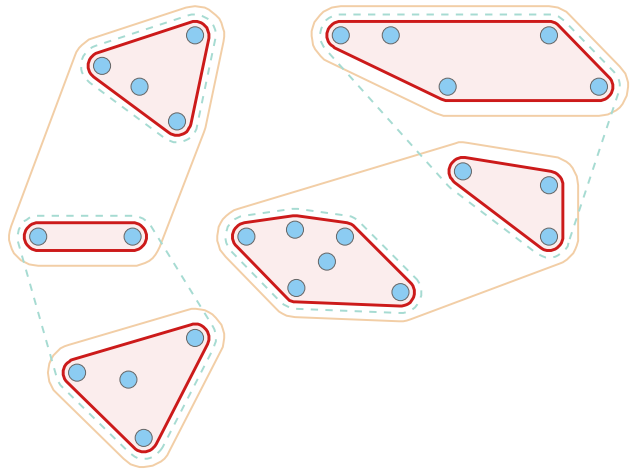
- Sélection fondée sur des connaissances complémentaires
- Étude préliminaire des données : histogrammes, ...
- Répétition de la méthode N fois $\Rightarrow$ N partitions et sélection de la partition ayant la plus faible inertie intra-classe
- Avec les formes fortes :
  - Répétition de la méthode N fois
  - Détermination des formes fortes : on regroupe les individus appartenant toujours aux mêmes classes
  - Initialisation des noyaux $=$ centres de gravité des meilleures formes fortes

Formes fortes

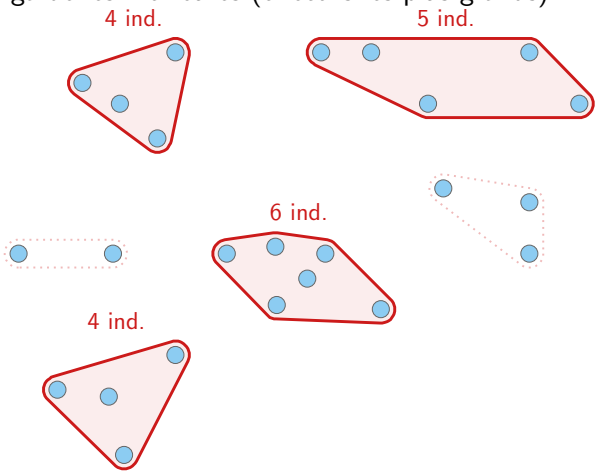
1 Deux exécutions de K-means ($K = 4$)



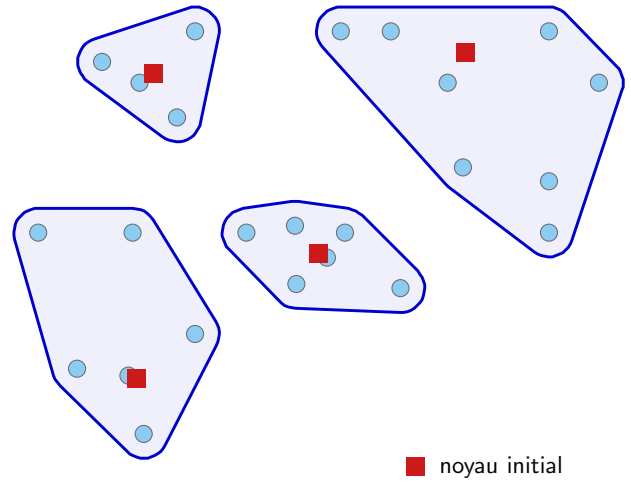
2 Formes fortes : individus toujours ensemble



3 On garde les meilleures (effectifs les plus grands)



4 Noyaux initiaux = centres ⇒ partition finale



Chapitre 2 - Méthodes de type K-means et DBSCAN

1. Méthodes de type K-means
2. Méthodes de type K-médoïdes
3. Choix des noyaux initiaux
4. Choix du nombre de classes
5. DBSCAN

Choix de K

- Pour chaque valeur de $K \in \{2, \dots, K_{\text{max}}\}$, on obtient une classification et on sélectionne finalement celle où on observe un saut important de l'inertie intra-classe ("coude")
- Choix par d'autres indices basés sur les inerties intra- et inter-
- Choix par maximisation d'un indice
  Ex: silhouette, statistique du Gap, ...
- Autres critères de type sélection de modèles (voir modèles de mélange)
- ...

Critères fondés sur les inerties

- **R-Square** :

$$K \mapsto RSQ(K) = 1 - \frac{I_{intra}(\mathcal{P}_K)}{I_{totale}} = \frac{I_{inter}(\mathcal{P}_K)}{I_{totale}}$$

On retient l'endroit où la courbe $K \mapsto \text{RSQ}(K)$ forme un coude.

- **Semi-Partial R-Square** :

$$K \mapsto SPRSQ(K) = \frac{I_{inter}(\mathcal{P}_K) - I_{inter}(\mathcal{P}_{K-1})}{I_{totale}}$$

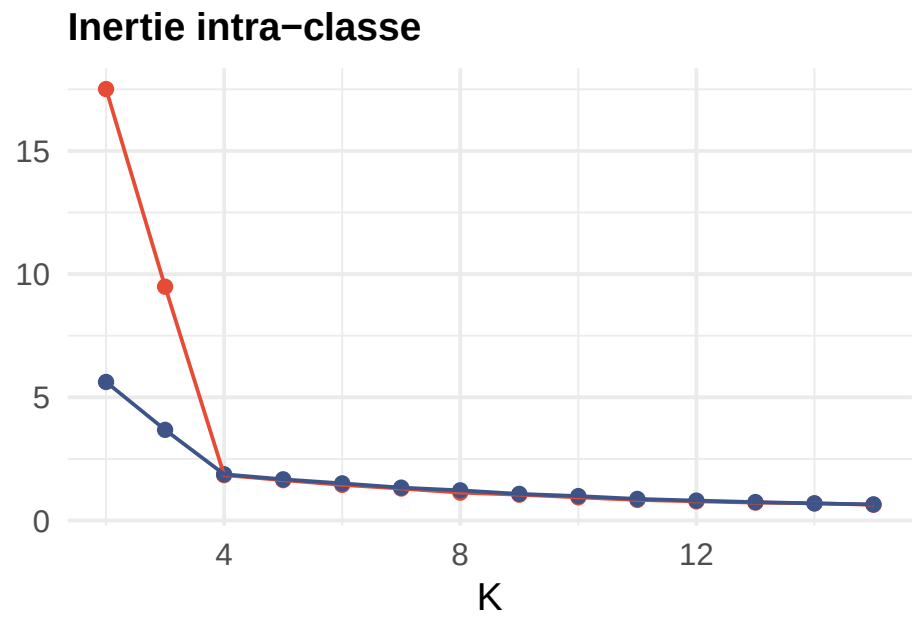
On retient l'endroit où on a la plus forte réduction du SPRSQ.

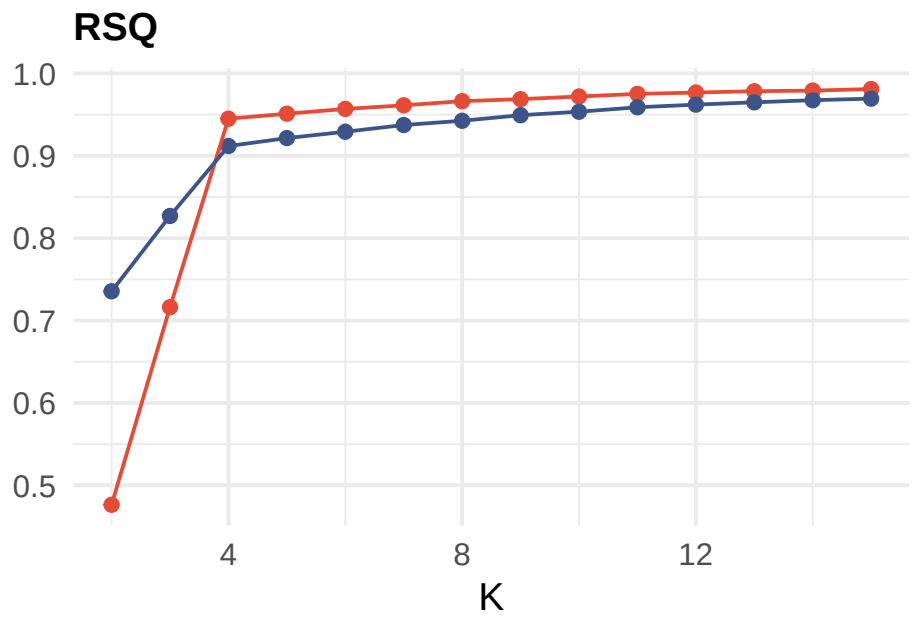
- **Calinski-Harabasz (CH)** :

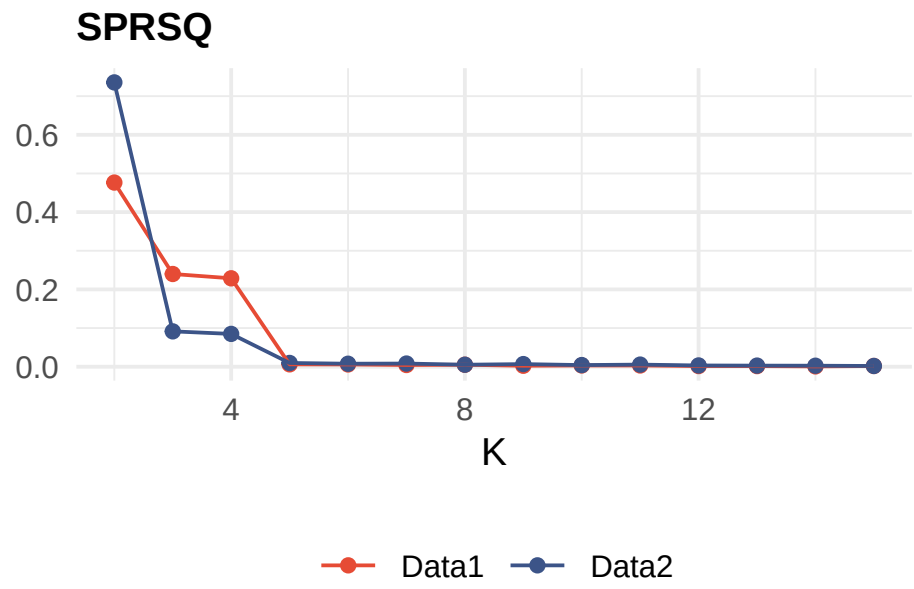
$$K \mapsto CH(K) = \frac{I_{inter}(\mathcal{P}_K)/(K-1)}{I_{intra}(\mathcal{P}_K)/(n-K)}$$

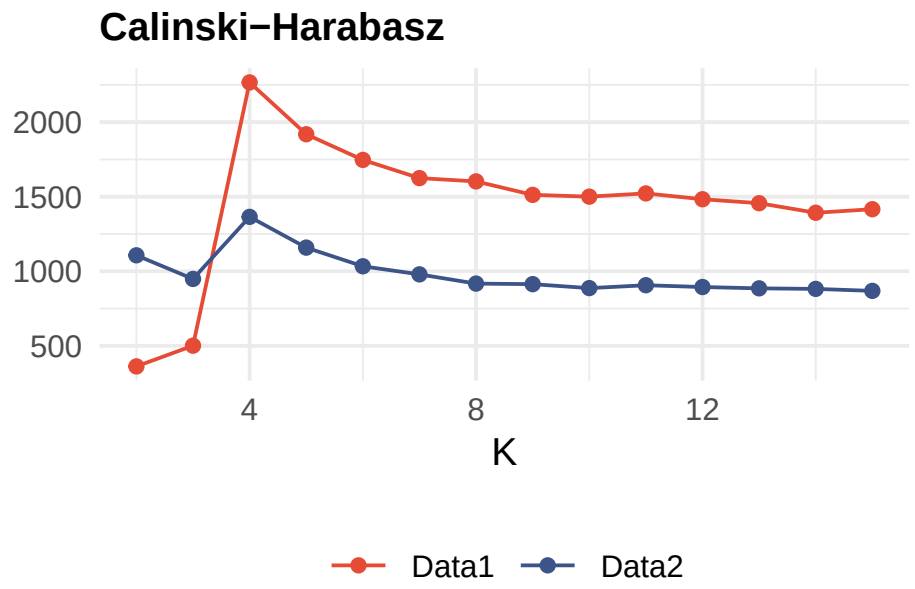
On cherche un pic sur cette courbe

Exemple sur les données simulées









Silhouette

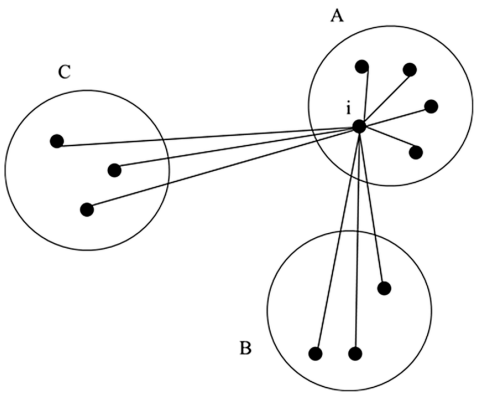
- Pour toute valeur $K \in \{2, \dots, K_{\max}\}$:
  - $\{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ classification de $\{1, \dots, n\}$
  - $\forall i \in \{1, \dots, n\}, \exists! k \in \{1, \dots, K\};\ i \in \mathcal{C}_k$

$$a(i) = \frac{1}{|\mathcal{C}_k|-1} \sum_{\substack{\ell \in \mathcal{C}_k \\ \ell \neq i}} d(x_i, x_\ell) \text{ et } b(i) = \min_{k' \neq k} \frac{1}{|\mathcal{C}_{k'}|} \sum_{\ell \in \mathcal{C}_{k'}} d(x_i, x_\ell)$$

$$s(i) = \frac{b(i)-a(i)}{\max(b(i),a(i))} \in [-1, 1] \text{ (et } s(i) = 0 \text{ si } |\mathcal{C}_k| = 1)$$

  - $S(K) = \frac{1}{n} \sum_{i=1}^{n} s(i)$

- Le nombre de classes retenu : $\hat{K} = \underset{2 \leq K \leq K_{\max}}{\text{argmax}}\ S(K)$



- avec  `silhouette(.)` [library(cluster)]
- avec  `silhouette_samples(.)`, `silhouette_score(.)` [scikit-learn]

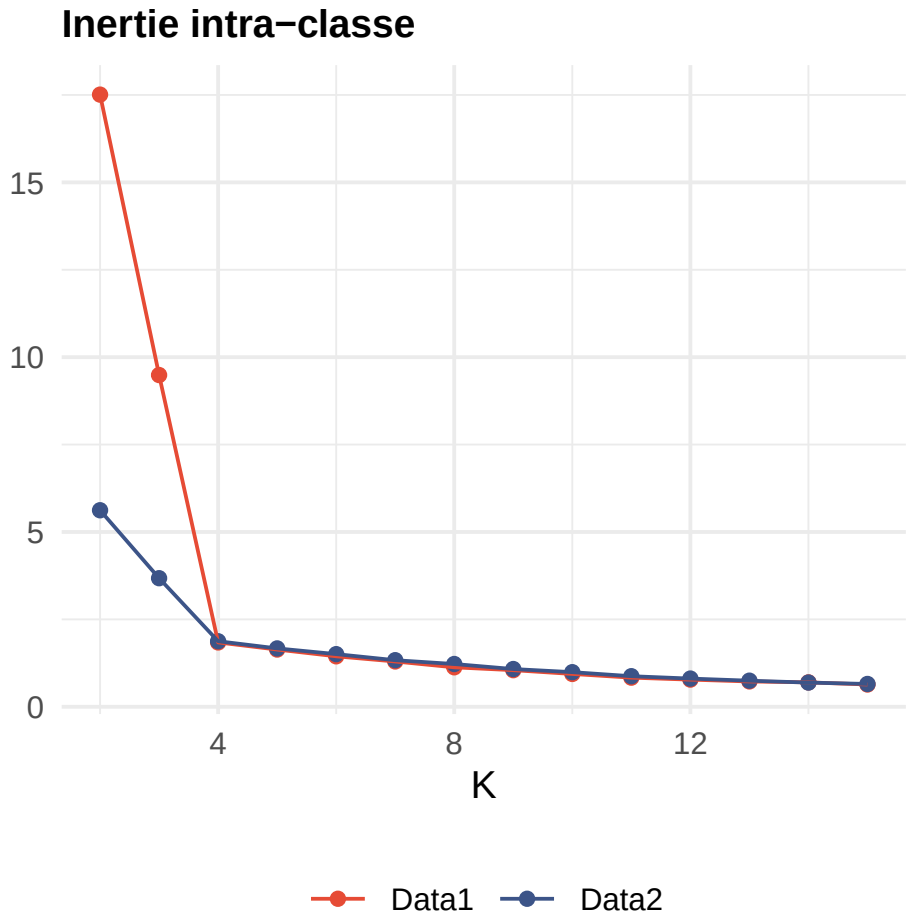
Exemple - Silhouette

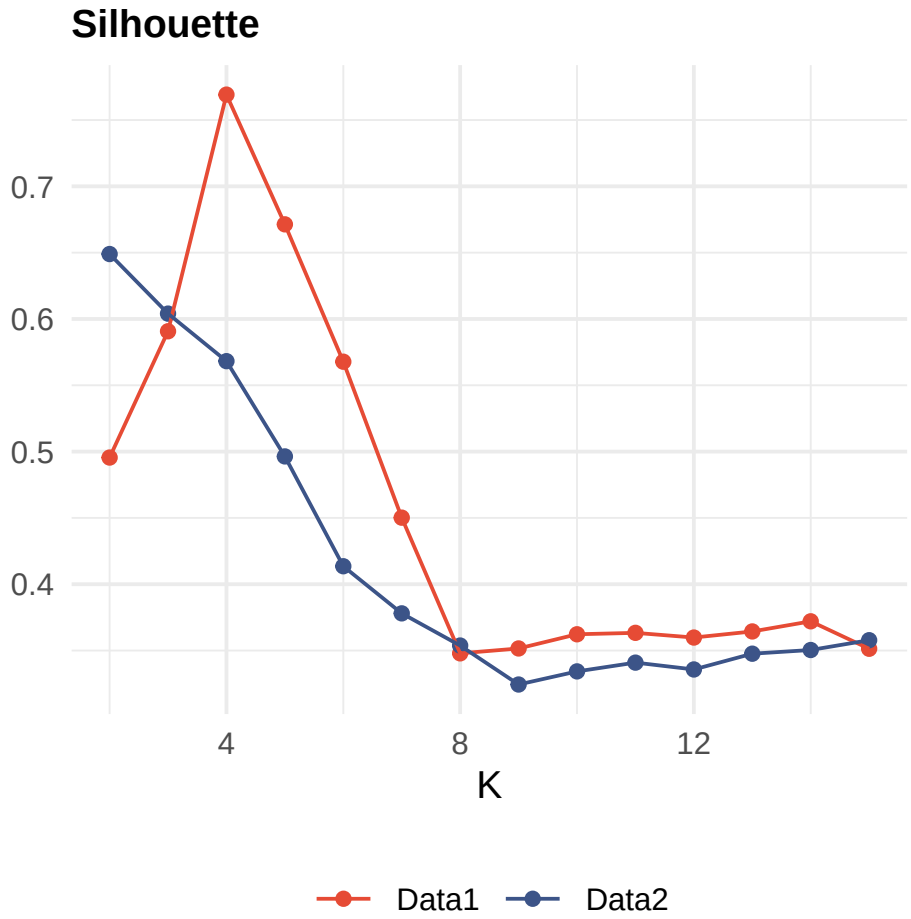
```
set.seed(1234)
Classif<-kmeans(datasimu1[,1:2],4,nstart=10)
aux<-silhouette(Classif$cl, daisy(datasimu1[,1:2]))
fviz_silhouette(aux,print.summary=TRUE)
```

```
  cluster size ave.sil.width
1       1   98          0.76
2       2  100          0.77
3       3  108          0.79
4       4   94          0.76
```



Exemple - Silhouette





Statistique du Gap (Tibshirani, Walther, and Hastie 2001)

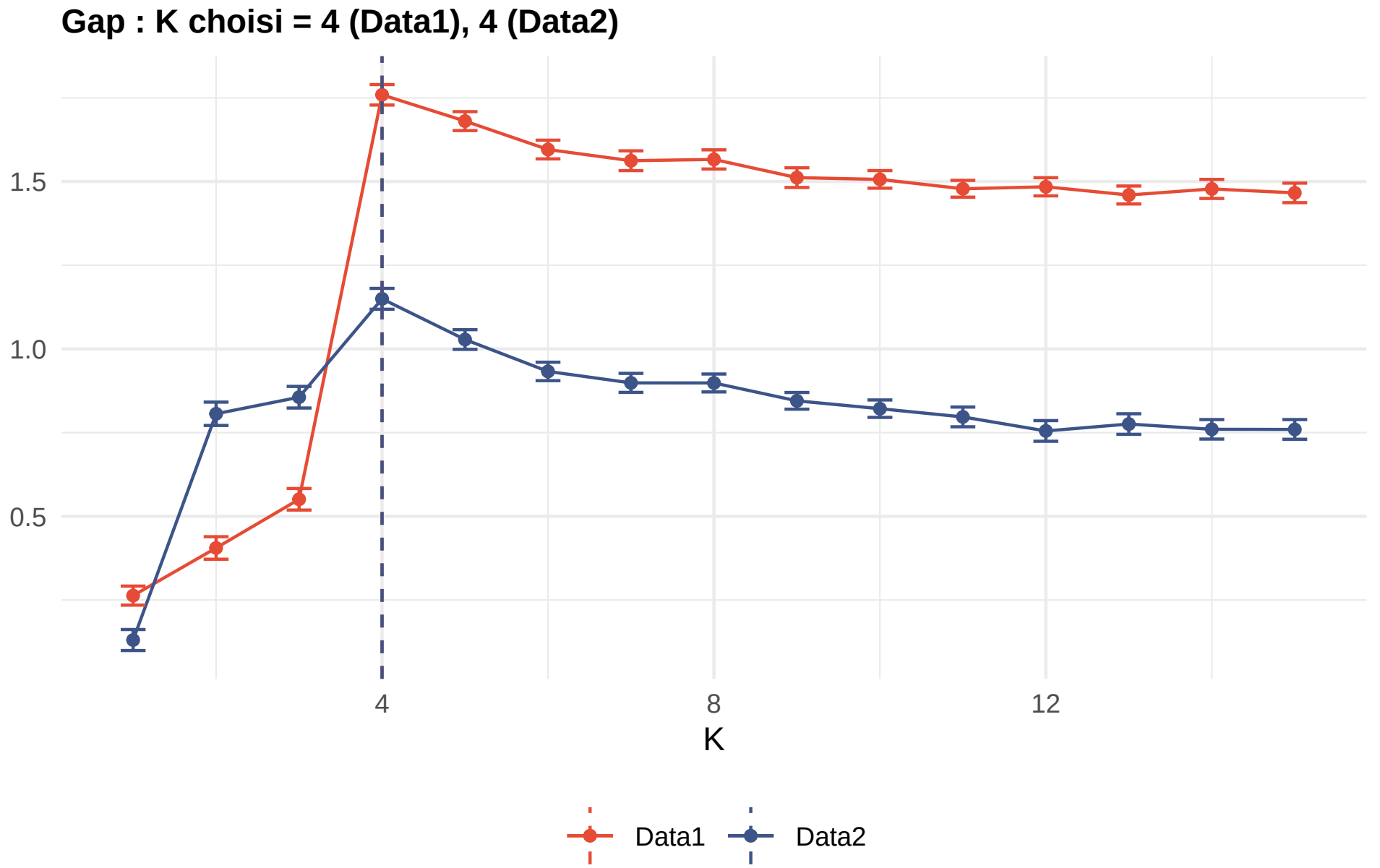
- Dispersion intra-classe de $\mathcal{P}_K$: $W_K = \sum_{k=1}^{K} \sum_{i \in \mathcal{C}_k} d(x_i, m_k)^2 = I_{intra}(\mathcal{P}_K)$
- Statistique du Gap : $\text{Gap}(K) = \mathbb{E}\left[\ln(W_K)\right] - \ln(W_K)$
  où l'espérance est sous une loi de référence **sans classes**
- En pratique : B jeux de référence simulés $\Rightarrow W^*_{K,1}, \ldots, W^*_{K,B}$

$$\text{Gap}(K) = \frac{1}{B} \sum_{b=1}^{B} \ln(W^*_{K,b}) - \ln(W_K), \qquad s_K = \text{sd}_K \sqrt{1 + 1/B}$$

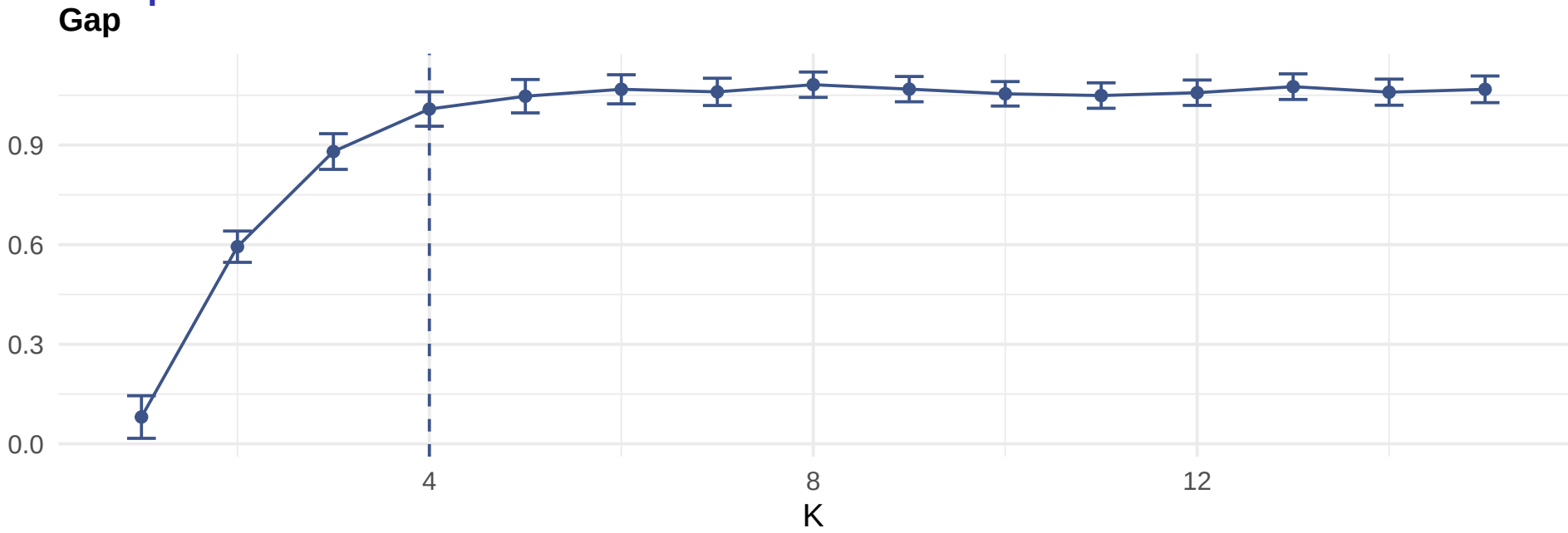
  avec sd_K l'écart-type des $\ln(W^*_{Kb})$

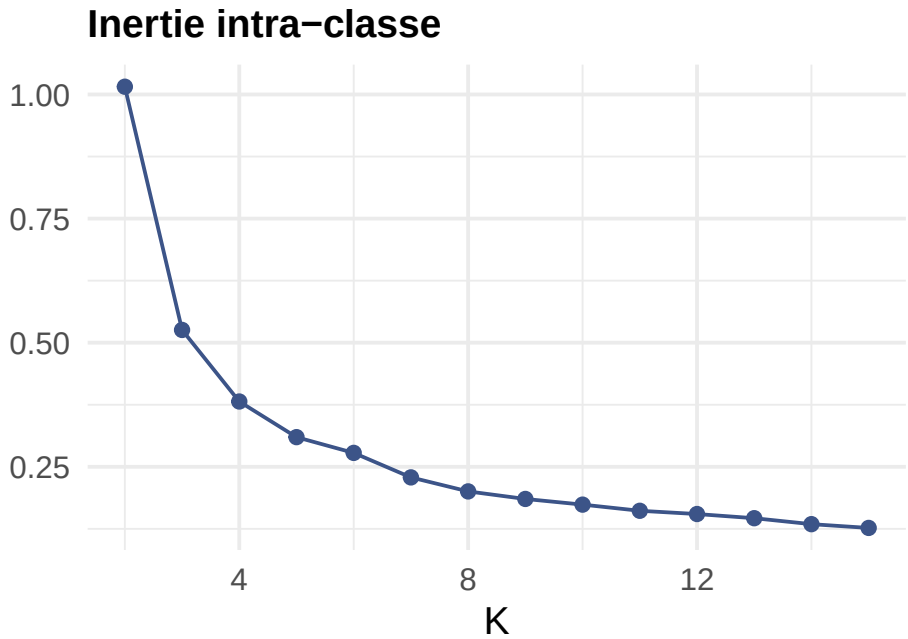
- Choix : $\hat{K} = \min\left\{K \ ; \ \text{Gap}(K) \geq \text{Gap}(K+1) - s_{K+1}\right\}$
- Avec  : `clusGap(x, FUN=kmeans, K.max=, B=, d.power=2)`
  puis `maxSE(gap, SE.sim, method="Tibs2001SEmax")`
- Avec  : github `milesgranger/gap_statistic`

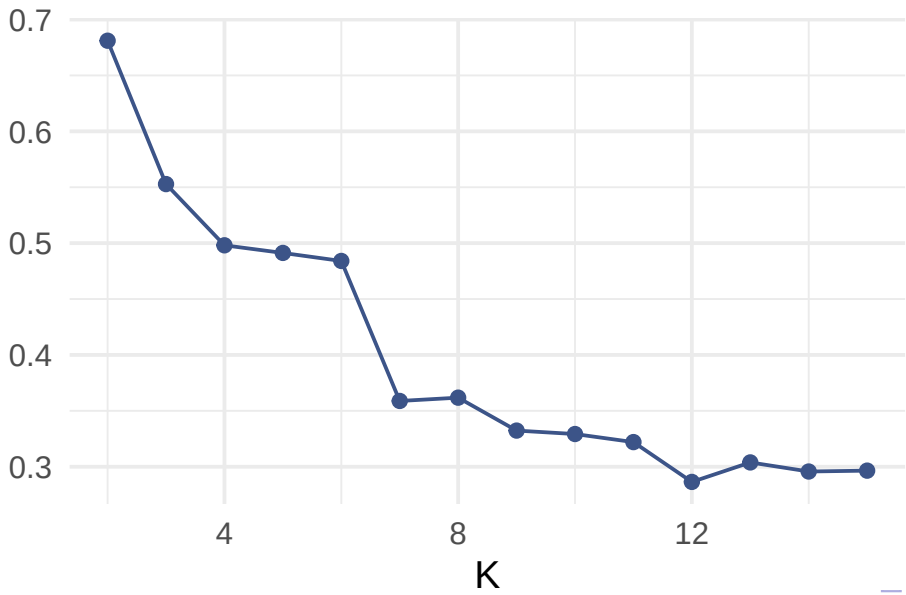
Exemple - Statistique du Gap



Exemple des Iris





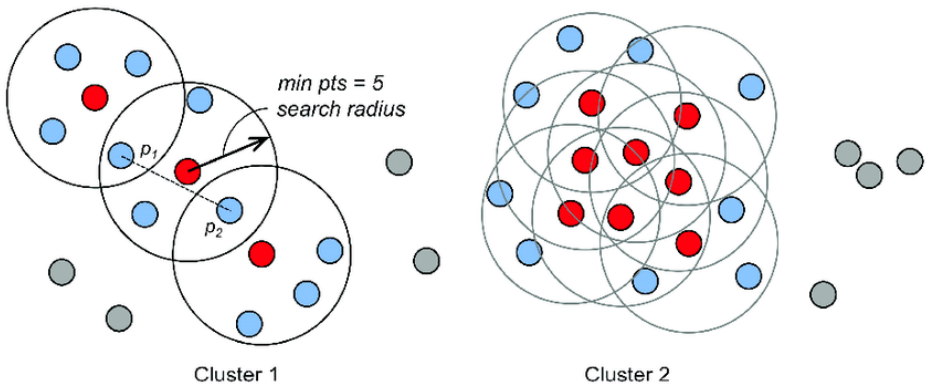


Chapitre 2 - Méthodes de type K-means et DBSCAN

1. Méthodes de type K-means
2. Méthodes de type K-médoïdes
3. Choix des noyaux initiaux
4. Choix du nombre de classes
5. DBSCAN

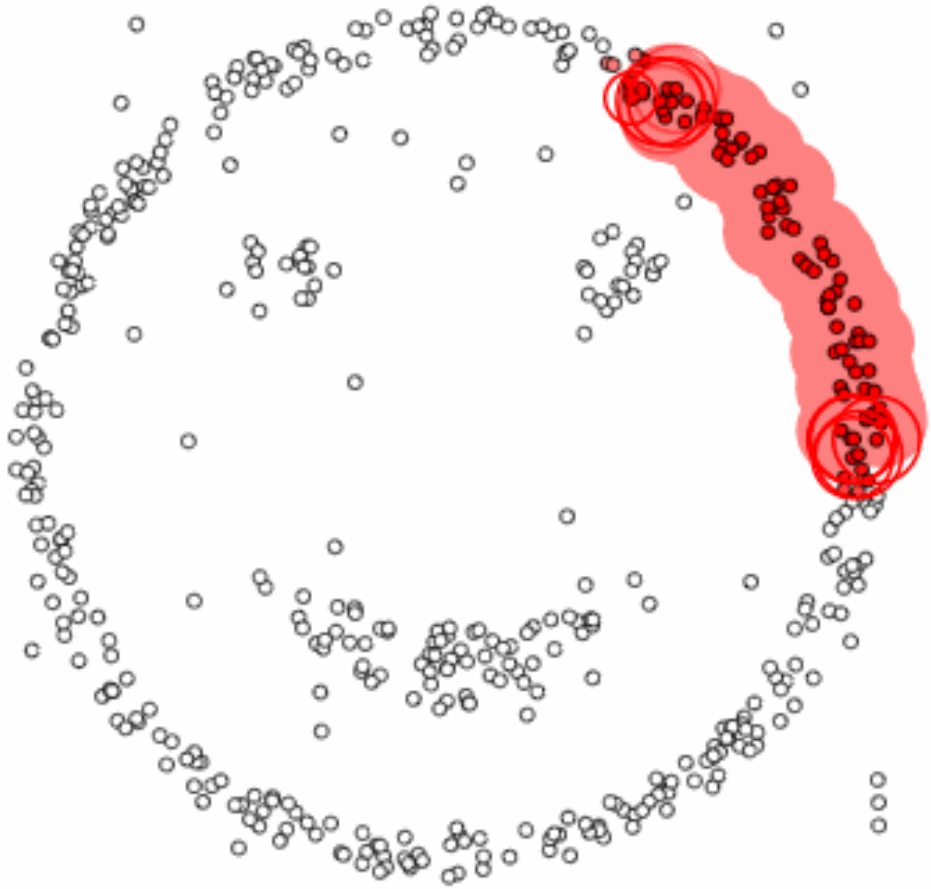
Principe (Ester et al. 1996)

- DBSCAN = Density-Based Spatial Clustering of Applications with Noise
- 2 paramètres :
  - la distance de voisinage ε
  - le nombre minimum de points minPts (le point lui-même compris)
- Idée : pour un indiv. donné, vérifier si son ε-voisinage contient au moins minPts points. Si oui, ce point fait partie d'une classe. On parcourt ensuite le ε-voisinage de proche en proche pour trouver l'ensemble des points de la classe.
- Usuellement, la distance euclidienne est utilisée



(A.Bonneau)

Animation



https://aaronscotthq.com/2020-05-28-scott_dbscan/

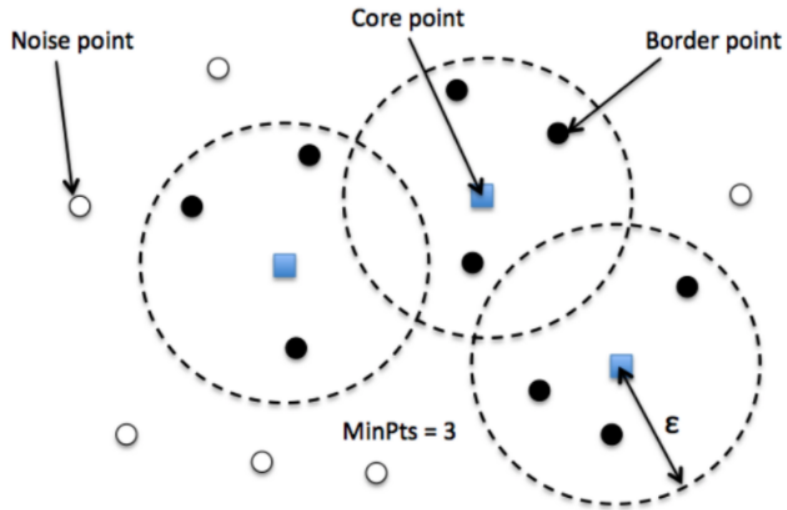
Catégories d'individus

A la fin de l'algorithme, l'individu x_i appartient à l'une de ces 3 catégories :

- Point central : son voisinage est dense

$$\sharp\{\ell \in \{1, \dots, n\};\ d(x_i, x_\ell) \leq \varepsilon\} \geq minPts$$

- Point frontière : appartient au ε-voisinage d'un point central mais n'est pas un point central
- Point aberrant (bruit) : n'est ni un point central, ni un point frontière (donc non classé)



Choix des paramètres

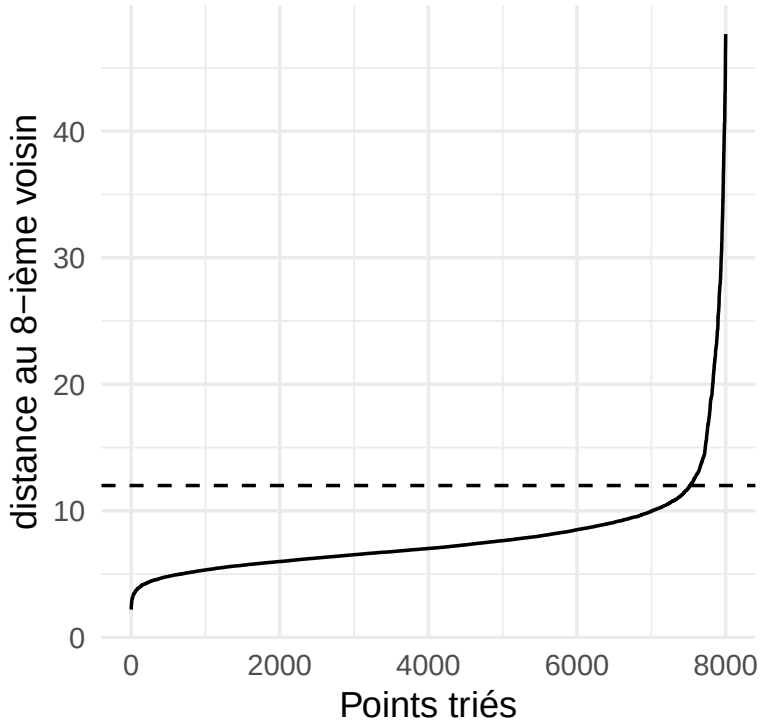
Choix du paramètre minPts :

- Il est préconisé de choisir au moins minPts $\geq p + 1$
- Pour des données avec du bruit (outliers), minPts$= 2p$ peut être utilisé mais il peut être nécessaire de choisir des valeurs plus grandes pour des données de grande dimension
- Dans d'autres sources, $\ln(n)$ est préconisé.

Choix des paramètres

Choix du paramètre ε :

- On trace le graphe de distance kNN :
  - Pour chaque point, on calcule sa distance à son k-ième plus proche voisin, avec $k = \mathrm{minPts} - 1$
  - On trace ces distances par ordre croissant
  - On choisit ε au niveau du "coude"
- Justification : x_i est un point central $\Leftrightarrow$ sa distance à son $(\mathrm{minPts} - 1)$-ième voisin est $\leq \varepsilon$

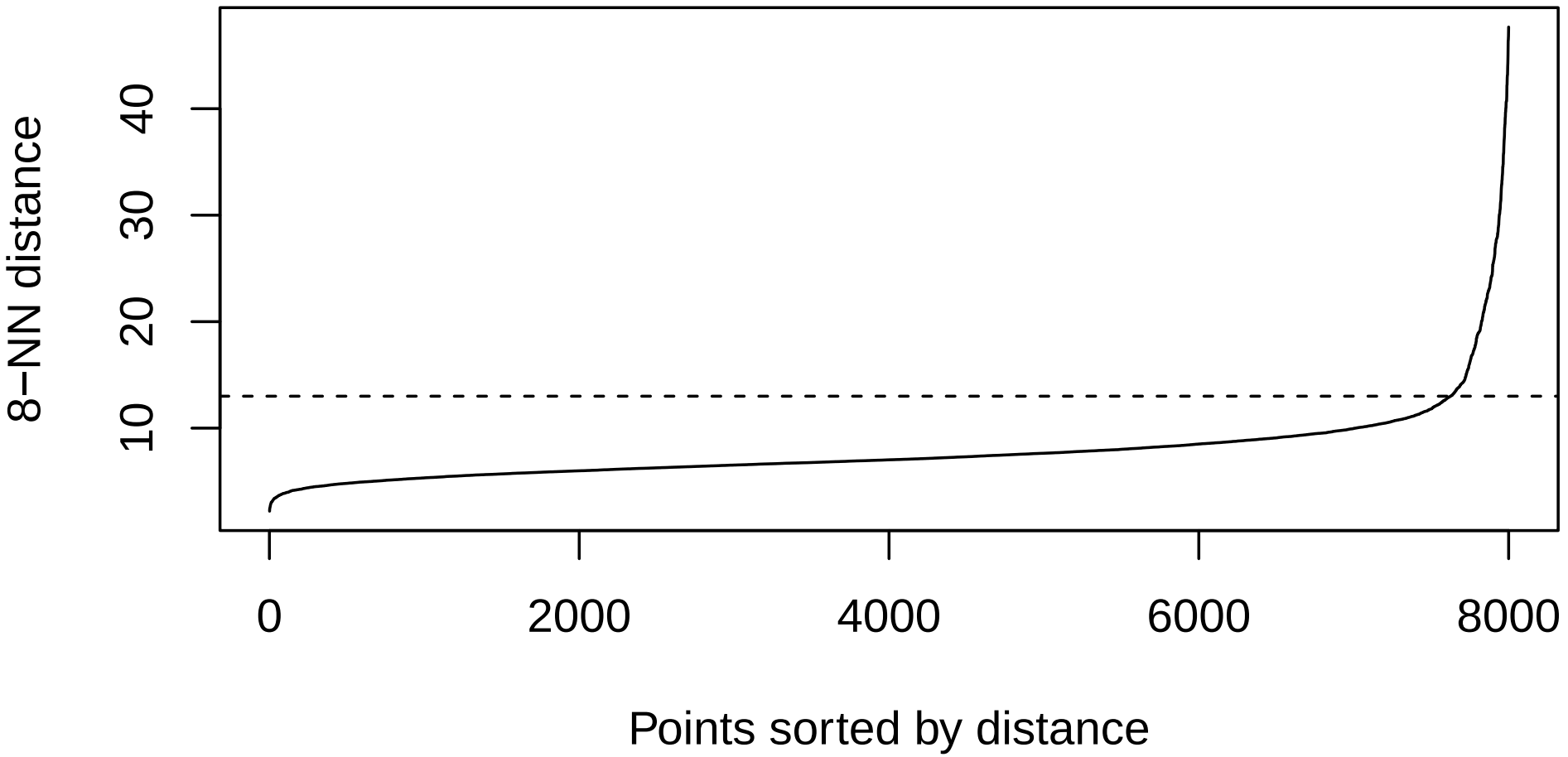


Quelques commandes

- Avec 
  - Fonction `dbscan()` de `library(fpc)`
  - Dans la `library(dbscan)`, les fonctions `dbscan()` et `kNNdistplot()`
- Avec 
  - Fonction `sklearn.cluster.DBSCAN()` de scikit-learn

Exemple avec 

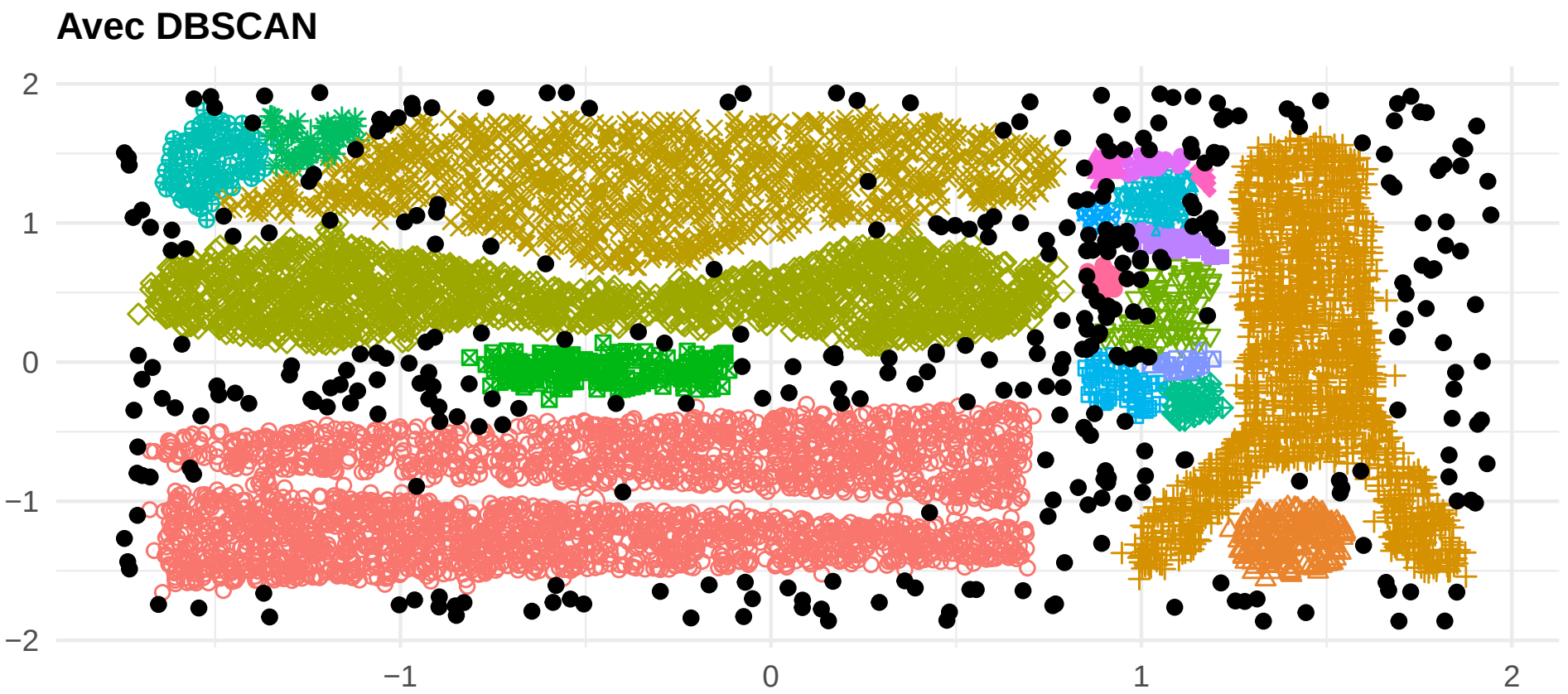
```
library(seriation)
data(Chameleon)
DS8<-chameleon_ds8
library(dbscan)
minPts <- round(log(nrow(DS8)))
kNNdistplot(DS8, minPts = minPts)
abline(h = 13, lty = 2)
```



Exemple avec 

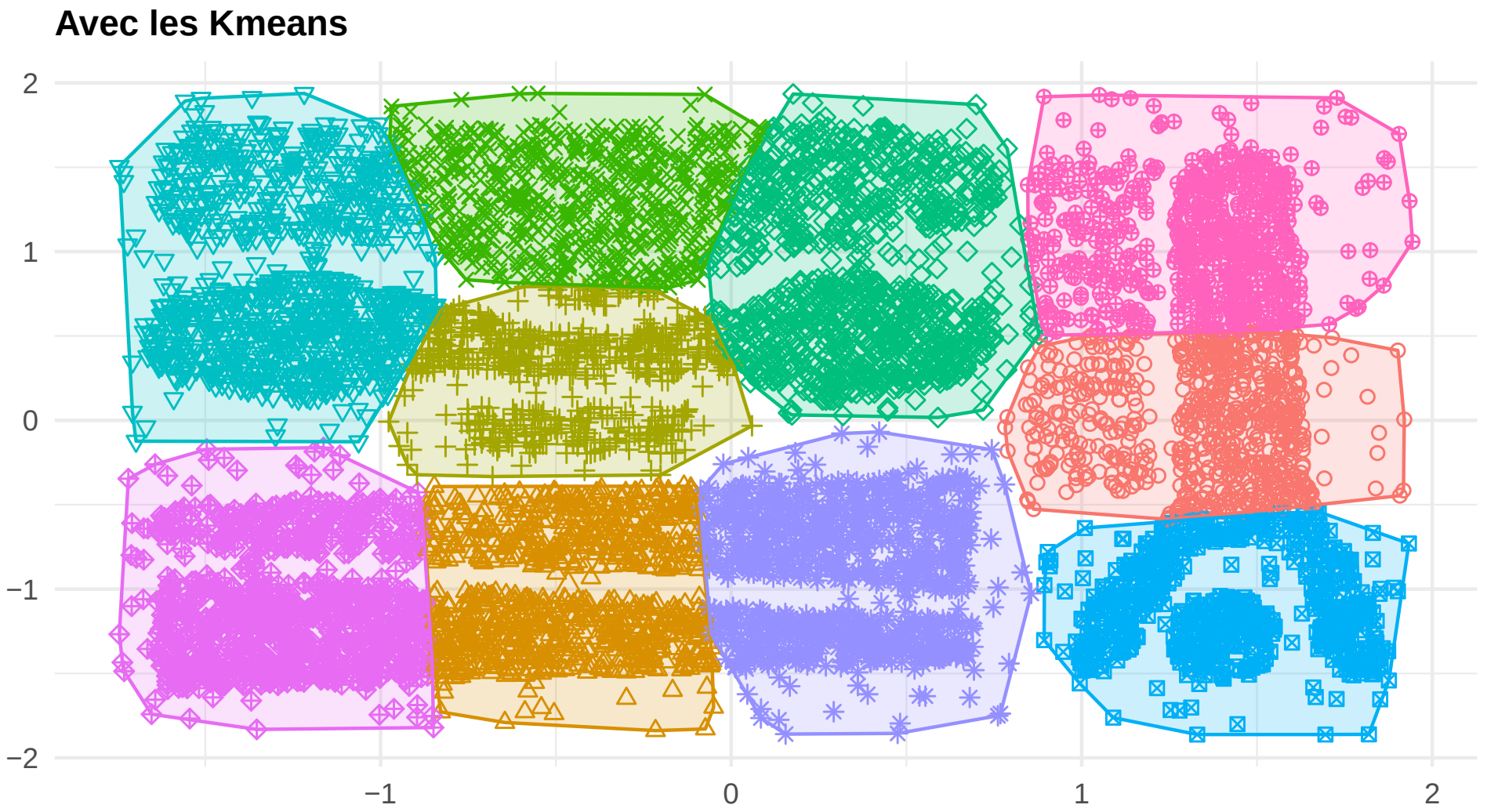
```
res.db <- dbscan::dbscan(DS8, 10, round(log(nrow(DS8))))
table(res.db$cluster)
```

```
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
357 2588 182 1391 1268 1576  79 177  55  31 120  31  42   9  16  30
 16  17  18  19
 15  13   7  13
```



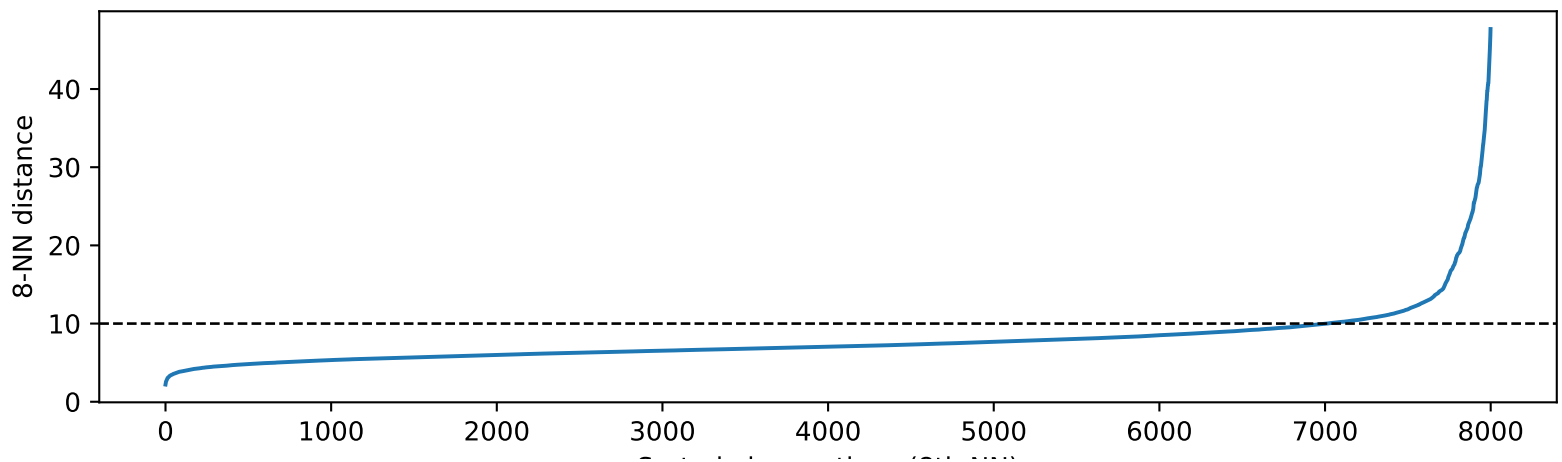
Exemple avec 

```
set.seed(1234)
fviz_cluster(kmeans(DS8,10), DS8, geom = "point")+
  themeCours+theme(legend.position="none")+
  xlab("")+ylab("")+ggtitle("Avec les Kmeans")
```



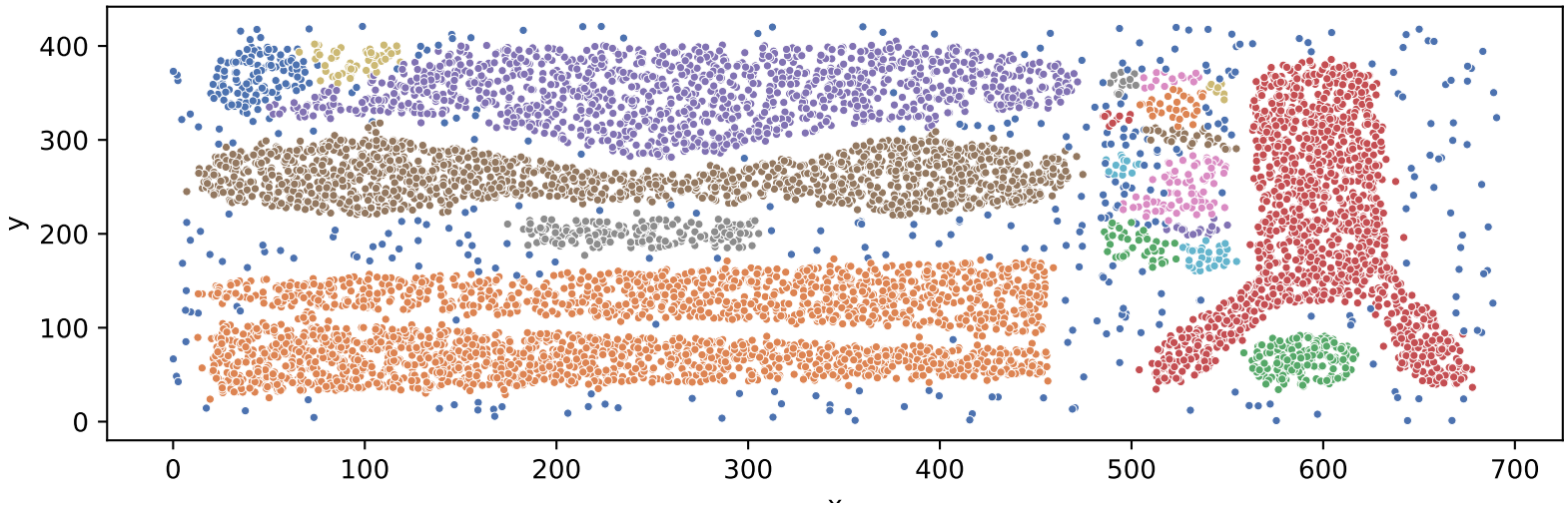
Exemple avec 

```
import numpy as np
from matplotlib import pyplot as plt
from sklearn.neighbors import NearestNeighbors
pyDS8=r.DS8
neighbors = NearestNeighbors(n_neighbors=9)
neighbors_fit = neighbors.fit(pyDS8)
distances, indices = neighbors_fit.kneighbors(pyDS8)
# distances[:,0] = 0 (le point lui-même) : distance au 8-ième voisin (minPts=9)
distancesk = np.sort(distances[:,8],axis=0)
```



Exemple avec 

```
from sklearn.cluster import DBSCAN
import seaborn as sns
import matplotlib.pyplot as plt
pyDS8=r.DS8
clusters = DBSCAN(eps=10, min_samples=9).fit(pyDS8)
fig = sns.scatterplot(data=pyDS8, x="x", y="y", hue=clusters.labels_, legend="brief", palette="deep",s=10)
plt.legend([],[], frameon=False)
plt.show()
```



References I

Diday, Edwin. 1971. "Une Nouvelle méthode En Classification Automatique Et Reconnaissance Des Formes La méthode Des Nuées Dynamiques." *Revue de Statistique Appliquée* 19 (2): 19–33.

Ester, Martin, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise." In *Kdd*, 96:226–31. 34.

Forgy, Edward W. 1965. "Cluster Analysis of Multivariate Data: Efficiency Versus Interpretability of Classifications." *Biometrics* 21: 768–69.

Huang, Zhexue. 1997. "Clustering Large Data Sets with Mixed Numeric and Categorical Values." In *Proceedings of the First Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD)*, 21–34.

———. 1998. "Extensions to the k-Means Algorithm for Clustering Large Data Sets with Categorical Values." *Data Mining and Knowledge Discovery* 2 (3): 283–304.

Kaufman, L., and P. J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.

References II

McQueen, James. 1967. "Some Methods for Classification and Analysis of Multivariate Observations." In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967*, 281–97.

Tibshirani, Robert, Guenther Walther, and Trevor Hastie. 2001. "Estimating the Number of Clusters in a Data Set via the Gap Statistic." *Journal of the Royal Statistical Society. Series B. Statistical Methodology* 63 (2): 411–23.